

Checkpoint — Training Data Preparation & Autoencoder Experiments

Yassine — Amen Bank Internship

June 19–20, 2026

1 Training Data Preparation Pipeline

1.1 Warm-Up Filtering

The first 21 days of enriched events (January 1–21) are excluded from all training, validation, and test sets. These events lack mature profile snapshots, producing distorted z-scores (mean ~ 438 , std $\sim 2,258$). Filtering occurs at training preparation time, not in the enrichment script, preserving `batch_enrichment.py` as a general-purpose artifact.

After filtering: 299,630 events retained from original 340,839.

1.2 Temporal Split

A chronological split prevents data leakage in time-series features (z-scores, cumulative amounts, drift metrics). No random shuffling.

Split	Day Range	Events	Anomalies
Train	Days 22–126	200,928	307
Validation	Days 127–153	49,382	43
Test	Days 154–180	49,320	64

Rationale: point-in-time integrity. A day-150 profile snapshot encodes knowledge of days 1–150; placing it in training while evaluating on day-30 events would leak future information.

1.3 Preprocessing

1. Drop constant columns: `currency` (always TND), `channel` (always guichet), `recent_events_buffer` (JSON structure, used by streaming inference only).
2. One-hot encode: `operation_type` (4 classes), `client_archetype` (5 classes).
3. StandardScaler fit on normal training data only, applied to all splits. Prevents anomaly influence on scaling parameters and ensures consistent transformation across splits.

Final feature dimension: 71 columns.

1.4 Autoencoder Dataset

- **Train:** 200,621 events (normal only, anomalies excluded)
- **Val:** 49,382 events (43 anomalies) — used for threshold calibration
- **Test:** 49,320 events (64 anomalies) — held out for final evaluation

Both models (Autoencoder and LSTM) train exclusively on normal data. Anomalous events are reserved for validation (threshold calibration, hyperparameter tuning) and test (final evaluation). This follows from the unsupervised anomaly detection paradigm: the model learns to reconstruct/predict “normal,” and deviations from normal signal anomalies.

1.5 LSTM Dataset

Sliding window of 50 events per client (stride 1). Each sample: (sequence of 50 events, target event #51). Split assignment based on the **target event’s timestamp**, not the window’s.

Split	Sequences	Anomalous Targets
Train	17,156	0 (all-normal windows enforced)
Validation	25,939	3
Test	37,457	33

Key design decision: LSTM sequences span across split boundaries. Context events (positions 1–50) may originate from the training period while the target event falls in validation or test. This mirrors production behavior, where the LSTM buffer always contains the client’s most recent 50 events regardless of calendar boundaries.

Initial implementation restricted all 51 events to the same split, yielding only 4 validation and 6 test sequences — unusable. The cross-boundary approach resolved this.

Training constraint: all 51 events (window + target) must be normal. If any anomalous event falls within the window, the sample is excluded to prevent the LSTM from learning anomalous patterns as “normal.”

2 Autoencoder Architecture

2.1 Model Design

Symmetric encoder-decoder with configurable depth and bottleneck:

Input(71) -> [Linear+ReLU+Dropout] x N -> Bottleneck
Bottleneck -> [Linear+ReLU+Dropout] x N -> Output(71)

Loss function: MSE (reconstruction error). Training objective: minimize reconstruction error on normal events. Anomaly scoring: reconstruction error magnitude — higher error indicates deviation from learned normal patterns.

2.2 Optuna Hyperparameter Search

Objective: maximize AUC-ROC on validation set. AUC was chosen over F_β because extreme class imbalance (0.087% anomaly rate in validation) causes F_β to plateau regardless of model quality — precision is dominated by the base rate.

Search space:

Hyperparameter	Range
Encoder layers	2–3
Bottleneck dimension	8–24 (step 4)
Dropout rate	0.1–0.4 (step 0.05)
Learning rate	$10^{-4} - 10^{-2}$ (log scale)
Batch size	{64, 128, 256}

Early stopping: patience 5 on validation reconstruction loss (normal events only). Training subsampled to 25% for search speed; final model retrained on full data. Threshold calibration via F_2 score ($\beta = 2$, recall-weighted) over percentiles 90.0–99.5 in steps of 0.5.

3 Autoencoder Experiments & Results

Three experimental rounds were conducted:

Experiment	Best AUC	Observation
71 features, mean MSE scoring	0.577	Weak but above random. Best architecture: 2 layers, bottleneck 24, dropout 0.1, lr ~ 0.001 .
34 behavioral features only (z-scores and flags removed)	0.411	Below random. Removing pre-computed anomaly signals worsened performance — behavioral profile features alone cannot distinguish anomalous events.
71 features, max/top- k scoring (Optuna over scoring method)	0.557	No improvement over mean scoring. Top- k and max methods did not resolve signal dilution.

3.1 Root Cause Analysis

The Autoencoder’s modest performance stems from a structural mismatch between the anomaly injection mechanism and the enriched feature representation:

1. Synthetic anomalies modify 1–2 raw event fields (amount, operation_type).
2. Enriched features are predominantly profile-based aggregates computed from hundreds of historical events.
3. A single anomalous event barely shifts these aggregates, so the enriched feature vector for an anomalous event is nearly identical to a normal one.
4. The reconstruction error is therefore similar for both, yielding AUC near random.

This is not a model architecture limitation — it reflects the fundamental difficulty of point-anomaly detection in profile-smoothed feature spaces. The finding is consistent with the literature: Autoencoders excel at detecting events that are individually unusual in the feature space, but struggle when anomaly signal is distributed across temporal context.

3.2 Decision

Accept the Autoencoder (AUC 0.577, 71 features, mean scoring) as a modest complementary signal. The pre-computed features (z-scores, boolean flags) provide stronger individual-event anomaly detection and will feed directly into the Decision Service. The LSTM, which detects *sequence-level* pattern breaks, is expected to provide stronger anomaly signal for this data structure.

Saved artifact: `models/autoencoder.pt` containing model state, hyperparameters, threshold, and validation metrics.

4 Key Design Decisions

Decision	Rationale
D23	Temporal split (chronological, not random) to prevent data leakage in time-series enriched features.
D24	StandardScaler fit on normal training data only to preserve anomaly extremity in scaled space.

D25	LSTM sequences span split boundaries; split assignment by target event timestamp. Mirrors production inference where the buffer contains historical events.
D26	AUC-ROC as Optuna objective for extreme-imbalance scenarios; F_β reserved for post-hoc threshold calibration.
D27	Autoencoder accepted at AUC 0.577 as complementary signal; pre-computed features (z-scores, flags) serve as primary point-anomaly indicators via Decision Service.

5 Project Structure Update

```
src/
  training/
    __init__.py
    data_prep.py      # Warm-up filter, temporal split, scaling,
                      # AE + LSTM dataset construction
    autoencoder.py    # Model, Optuna search, final training, save
data/
  training/
    autoencoder.npz   # AE train/val/test arrays
    lstm.npz          # LSTM sequences/targets/labels
    feature_cols.csv  # Feature column names
models/
  autoencoder.pt      # Trained model + threshold + params
```

6 Next Session Plan

1. Discuss Autoencoder findings and decide whether to iterate further or proceed.
2. Design and implement LSTM architecture: 6 multi-task prediction heads (operation_type, amount_bucket, branch known/new, employee known/new, beneficiary known/new, emitter known/new).
3. LSTM training with Optuna hyperparameter search.
4. Score fusion strategy: combining Autoencoder reconstruction error + LSTM prediction error + pre-computed rule features.